

Sistema de Gestión de Estados (Máquina de Estados)

Este documento describe la implementación de la máquina de estados en la clase principal `Main.java` para gestionar el flujo del ciclo de vida del juego.

Objetivo

Separar la lógica de menús, juego activo y pantallas finales para evitar que los procesos (como disparos o colisiones) se ejecuten cuando el jugador no está activamente en la partida.

Componentes del Sistema

1. Definición de Estados

Se utiliza un `enum` para centralizar los estados posibles:

- MENU: Pantalla de inicio, espera interacción para empezar.
- JUGANDO: Estado activo donde el `Controlador` gestiona la lógica y las entidades.
- GAME_OVER / VICTORIA: Pantallas de fin de juego con opción de retorno al menú.

2. Estructura del `render()`

El método `render()` ahora actúa como un distribuidor de tareas mediante un bloque `switch`:

java

```
@Override
public void render() {
    ScreenUtils.clear(0.15f, 0.15f, 0.2f, 1f);

    switch (estadoActual) {
        case MENU:
            // Dibujar menú y esperar click para empezar
            if (Gdx.input.justTouched()) estadoActual = Estado.JUGANDO;
            break;

        case JUGANDO:
            // Lógica y simulación activa
            Controlador.getInstance().simulaMundo(anchoPantalla, altoPantalla);
            Controlador.getInstance().pintar(batch, galeriaImágenes);
            break;

        case GAME_OVER:
            // Pantalla de derrota y reset
            if (Gdx.input.justTouched()) estadoActual = Estado.MENU;
```

```
        break;
    }
}
```

Ventajas

1. Orden y Rendimiento: El `Controlador` solo consume recursos de CPU en el estado `JUGANDO`.
2. Escalabilidad: Añadir nuevas pantallas (como una de "Pausa") solo requiere un nuevo valor en el `enum` y un `case` adicional.
3. Limpieza: Las texturas se gestionan en un único `HashMap` (`galeriaImagenes`), facilitando su liberación en el método `dispose()`.

Tareas Pendientes

- Conectar los métodos de comprobación de victoria/derrota del `Controlador` con el cambio de estado en el `render`.
- Implementar un método de reinicio de entidades en el `Controlador` al volver al menú.